CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

UNIT I

INTRODUCTION TO
DESIGN AND ANALYSIS OF ALGORITHMS

Lecture Notes for Engineering Students

B.E. / B.Tech | Computer Science & Engineering

SIRAGU

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

UNIT I — COURSE OVERVIEW

Topics Covered

This unit introduces the foundational concepts of algorithm design and analysis. The three main pillars are:

Topic Area	Sub-topics
Problem Solving	Programs & Algorithms, Problem Solving Aspects, Techniques
Algorithm Analysis	Time & Space Complexity, Asymptotic Notations, Recurrence Relations
String Matching	Interpolation Search, Naive, Rabin-Karp, KMP Algorithms

SIRAGU

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

SECTION 1: PROBLEM SOLVING

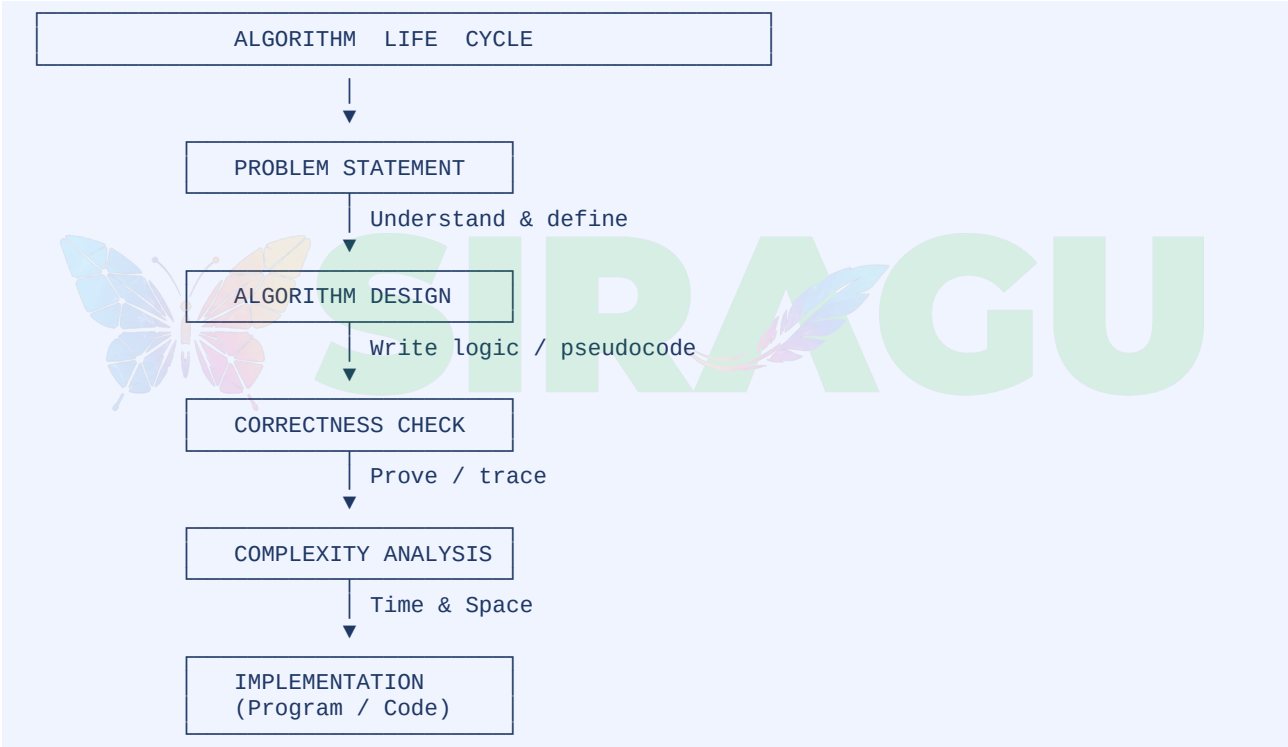
1.1 Programs and Algorithms

What is an Algorithm?

An algorithm is a step-by-step procedure or a set of well-defined instructions to solve a particular problem in a finite amount of time.

Key Properties: Finiteness · Definiteness · Input · Output · Effectiveness

Figure 1.1: Algorithm Life Cycle



Algorithm vs Program

Algorithm	Program
Language-independent (pseudocode)	Written in a specific language (C, Java, Python)
Exists at design phase	Exists at implementation phase
Does not run on a machine	Runs on a computer
Focuses on logic	Focuses on syntax and execution

Pseudocode Example — Linear Search

```
Algorithm LinearSearch(A, n, key)
  Input : Array A of n elements, search key
  Output: Index of key in A, or -1 if not found
  for i ← 0 to n-1 do
      if A[i] = key then
          return i
  return -1
```

1.2 Problem Solving Aspects

Understanding the Problem

Before writing an algorithm, you must clearly understand:

- What are the inputs? (type, size, constraints)
- What is the expected output?
- Are there any special cases or edge conditions?
- What is the performance requirement?

Problem Solving Steps

Figure 1.2: Six-Step Problem Solving Process



1.3 Problem Solving Techniques

Several general strategies are used in algorithm design:

Technique	Core Idea	Example
Brute Force	Try all possibilities exhaustively	Naive String Match, Selection Sort
Divide & Conquer	Split problem, solve sub-problems, combine	Merge Sort, Binary Search
Dynamic Programming	Store results of sub-problems (memoization)	Fibonacci, Knapsack
Greedy	Pick the best local choice at each step	Dijkstra, Huffman Coding
Backtracking	Build solution step-by-step, undo bad choices	N-Queens, Sudoku
Branch & Bound	Explore best-first with pruning	Travelling Salesman Problem

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

SECTION 2: ALGORITHM ANALYSIS

2.1 Time and Space Complexity

Why Analyse?

Two algorithms solving the same problem may differ greatly in speed and memory usage. We measure complexity to compare them fairly, independent of hardware.

Time Complexity: How the running time grows as the input size n increases.

Space Complexity: How much memory the algorithm uses as n increases.

Counting Operations — Simple Example

Consider summing an array of n elements:

```
sum ← 0                    → 1 operation
for i ← 1 to n do          → n iterations
    sum ← sum + A[i]       → 1 operation per iteration
return sum                 → 1 operation
```

Total operations ≈ 1 + n + 1 = n + 2 → O(n)

Frequency Count Method

Figure 2.1: Counting Operations in Nested Loops

Code Fragment	Count
x = 5;	1
for(i=0; i<n; i++)	n
for(j=0; j<n; j++)	n^2
A[i][j] = i + j;	n^2
Total	$1 + n + 2n^2 \rightarrow O(n^2)$

Linear Space Complexity – O(n)

C++ Code:

```
#include<iostream>
using namespace std;

int main()
{
   int n;
   cin >> n;

   int arr[n];

   for(int i=0;i<n;i++)
   {
```

```
        cin >> arr[i];
    }

    return 0;
}
```

Space Analysis:
If:
n = 5
Memory:
arr[5]

If:
n = 1000

Memory:
arr[1000]

Array size grows according to input size.
Memory required depends on **n**.
Space Complexity:

Common Complexity Classes

Recognising these classes helps you choose the most efficient algorithm for the job.

	Time Complexity	Space Complexity
01	$O(1)$ Constant — instant	$O(1)$ Constant — fixed variables
02	$O(\log n)$ Logarithmic — binary search	$O(\log n)$ Logarithmic — recursion stack
03	$O(n)$ Linear — single loop	$O(n)$ Linear — input copy
04	$O(n \log n)$ Linearithmic — efficient sort	$O(n^2)$ Quadratic — 2D structures
05	$O(n^2)$ Quadratic — nested loops	
06	$O(2^n)$ Exponential — brute recursion	

SIRAGU SIRAG

2.2 Asymptotic Notations

Asymptotic notation describes the growth rate of an algorithm's complexity as $n \to \infty$, ignoring constants.

Big-O Notation O(f(n)) — Upper Bound (Worst Case)

We say T(n) = O(f(n)) if there exist constants c > 0 and n_0 such that:

$T(n) \leq c \cdot f(n)$ for all $n \geq n_0$

Simple meaning: The algorithm will NEVER be worse than f(n) (up to a constant factor).

Big-Omega Ω(f(n)) — Lower Bound (Best Case)

We say T(n) = Ω(f(n)) if there exist c > 0 and n_0 such that:

$T(n) \geq c \cdot f(n)$ for all $n \geq n_0$

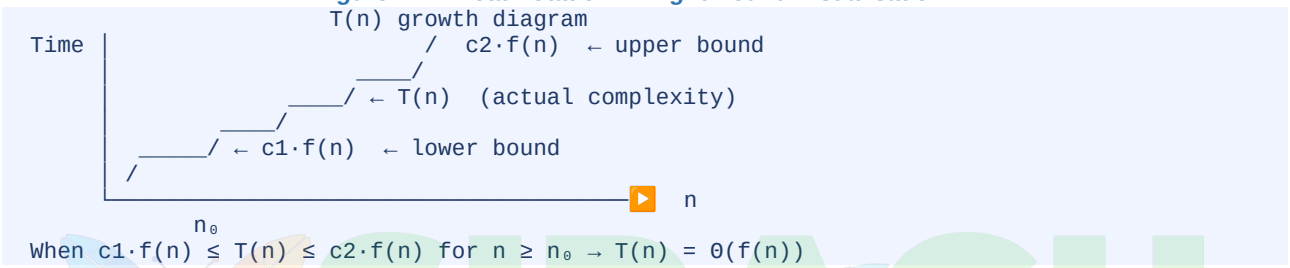
Simple meaning: The algorithm will ALWAYS take at least f(n) steps.

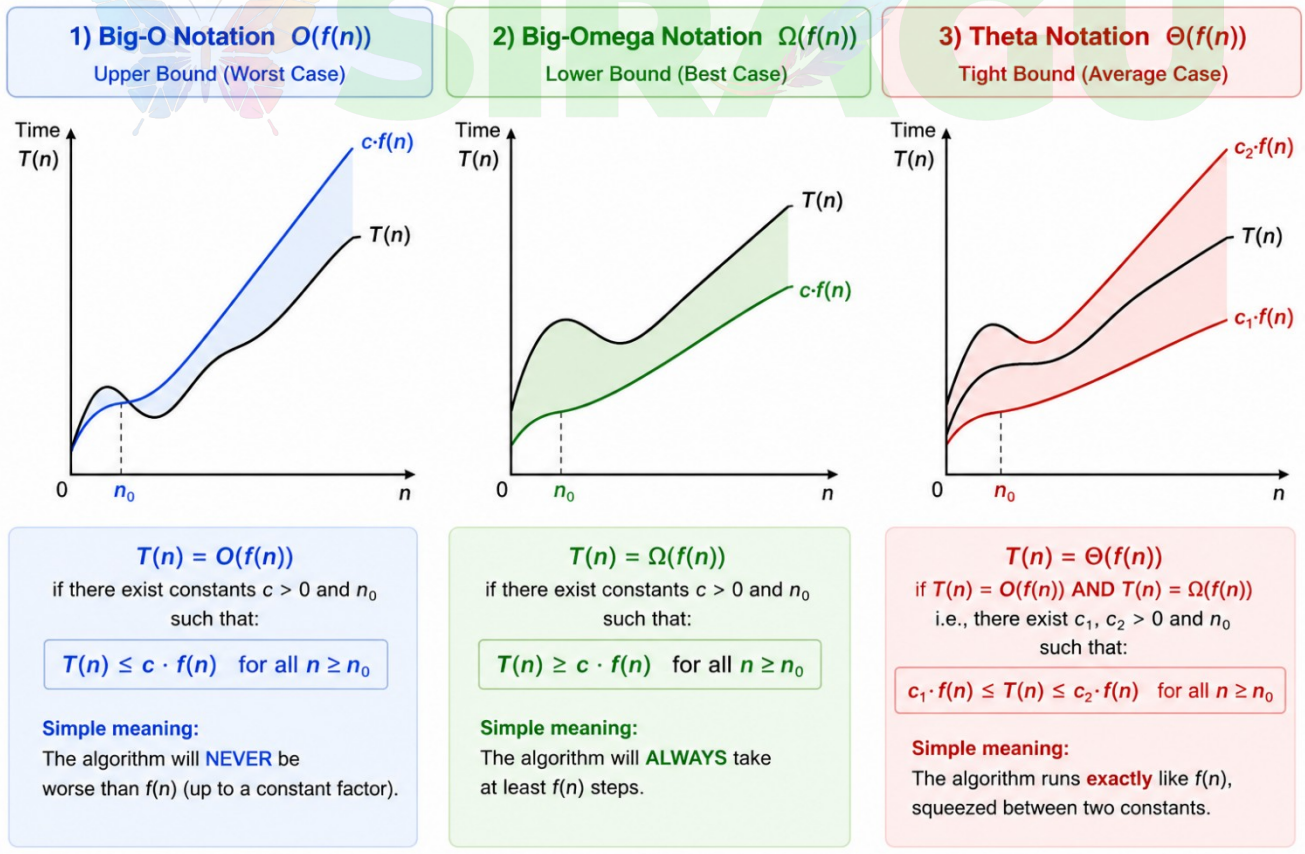
Theta-Θ(f(n)) — Tight Bound (Average Case)

We say T(n) = Θ(f(n)) if T(n) = O(f(n)) AND T(n) = Ω(f(n)).

Simple meaning: The algorithm runs exactly like f(n), squeezed between two constants.

Figure 2.2: Theta Notation — Tight Bound Visualisation





Common Complexity Classes (Best → Worst)

Notation	Name	Example Algorithm	n=100 (approx ops)
O(1)	Constant	Array index access	1
O(log n)	Logarithmic	Binary Search	7
O(n)	Linear	Linear Search	100
O(n log n)	Linearithmic	Merge Sort, Heap Sort	664
$O(n^2)$	Quadratic	Bubble / Insertion Sort	10,000
$O(n^3)$	Cubic	Matrix Multiplication (naive)	1,000,000
$O(2^n)$	Exponential	Subset Enumeration	$\sim 10^{30}$

Properties of Asymptotic Notations

Property	Statement
Reflexivity	$f(n) = O(f(n))$ — every function is its own upper bound
Transitivity	If $f = O(g)$ and $g = O(h) \rightarrow f = O(h)$
Symmetry (Θ)	$f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$
Transpose Symmetry	$f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$
Sum Rule	$O(f) + O(g) = O(\max(f, g))$
Product Rule	$O(f) \times O(g) = O(f \times g)$

SIRAGU

2.3 Best, Worst, and Average Case Analysis

For a given algorithm, the time taken may differ for different inputs of the same size. We distinguish three cases:

Figure 2.3: Case Analysis for Linear Search

```
BEST CASE  ——— smallest input that causes fewest operations
————————————————————————————————————————————————————————
Example — Linear Search: key found at FIRST position → O(1)

WORST CASE ——— input that causes MAXIMUM operations
————————————————————————————————————————————————————————
Example — Linear Search: key not in array → O(n)

AVERAGE CASE — expected operations over all possible inputs
————————————————————————————————————————————————————————
Example — Linear Search: key found at middle → O(n/2) = O(n)
```

Algorithm	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n/2) = O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$

2.4 Recurrence Relations — Substitution Method

Recurrence relations express the running time T(n) of a recursive algorithm in terms of T(n/2), T(n-1), etc.

Substitution Method — Step by Step

1. Guess the form of the solution (e.g., T(n) = O(n log n))
2. Substitute the guess back into the recurrence
3. Use mathematical induction to prove the bound

Example 1 — Binary Search Recurrence

```
T(n) = T(n/2) + 1,   T(1) = 1
```

Guess: T(n) = O(log n)

Substitute: T(n) = T(n/2) + 1

= T(n/4) + 1 + 1 = T(n/4) + 2

= T(n/8) + 3

= ... = $T(n/2^k) + k$

Stop when $n/2^k = 1$ → $k = \log_2 n$

Therefore: $T(n) = 1 + \log_2 n = O(\log n)$ ✓

Example 2 — Merge Sort Recurrence

$$T(n) = 2 \cdot T(n/2) + n, \quad T(1) = 1$$

Guess: T(n) = O(n log n)

Substitute: T(n) = 2·T(n/2) + n

= 2[2·T(n/4) + n/2] + n = 4·T(n/4) + 2n

= 8·T(n/8) + 3n

= $2^k \cdot T(n/2^k) + kn$

At k = $\log_2 n$: T(n) = n·T(1) + n·$\log_2 n$ = n + n log n = O(n log n) ✓

Figure 2.4: Recursion Tree for Merge Sort Recurrence

```
Recursion Tree for T(n) = 2T(n/2) + n
──────────────────────────────────────────────────────────
Level 0:            T(n)              cost = n
                   /    \
Level 1:       T(n/2) T(n/2)          cost = n/2 + n/2 = n
               /  \    /  \
Level 2:    T(n/4)..T(n/4)            cost = n
              ...
Level log n: T(1) T(1) ... T(1)        cost = n
──────────────────────────────────────────────────────────
Total cost = n × log n levels = O(n log n)
```

Master Theorem

For T(n) = aT(n/b) + f(n), a ≥ 1, b > 1, compute critical exponent E = log_b(a):

Case	Condition	Result
Case 1	f(n) = O(n^(E−ε)) for some ε > 0	T(n) = Θ(n^E)
Case 2	f(n) = Θ(n^E)	T(n) = Θ(n^E · log n)
Case 3	f(n) = Ω(n^(E+ε)) and regularity cond.	T(n) = Θ(f(n))

Quick check: T(n)=2T(n/2)+n: a=2,b=2,E=$\log_2 2$=1. f(n)=n=Θ(n^1). Case 2 → T(n)=Θ(n log n). ✓

Substitution is lengthy.
For divide and conquer algorithms we use Master Theorem.

Definition:

Master Theorem provides a direct method to solve recurrence relations of the form:

Where:
a = Number of subproblems

b = Size reduction factor

f(n)= Extra work outside recursion

Example:
Merge Sort:

Here:
a = 2

b = 2

f(n)=n

Master Theorem Steps
Step 1:
Find:
a

b

f(n)

Step 2:
Calculate:

Step 3:
Compare:

with

SIRAGU

Three Cases of Master Theorem
Case 1
If:

Then:

Recursive work dominates.

Case 2
If:

Then:

Both are balanced.

Case 3
If:

Then:

Outside work dominates.

Master Theorem Example 1
Solve:

Find:
a=2

b=2

f(n)=n
Calculate:

Compare:
n = n
Case 2
Answer:

SIRAGU

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

SECTION 3: STRING MATCHING AND SEARCHING

3.1 Interpolation Search

Interpolation search is an improved variant of binary search designed for uniformly distributed sorted arrays. Instead of always going to the middle, it estimates where the key might be.

Formula for Position Estimate

Position Formula: pos = low + [(key - A[low]) / (A[high] - A[low])] × (high - low)

Intuition: If you are looking for page 800 in a 1000-page book, you open near the end — not the middle. Interpolation search does the same.

Figure 3.1: Interpolation Search Example

```
Sorted Array A: [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
Search key = 70

low = 0, high = 9
pos = 0 + [(70-10)/(100-10)] × (9-0)
    = 0 + [60/90] × 9
    = 0 + 0.667 × 9
    = 6   ← Direct hit! A[6] = 70  ✓

Binary Search would take 4 steps; Interpolation took 1 step!
```

Complexity of Interpolation Search

Case	Time Complexity	Condition
Best Case	O(1)	Key found at first estimated position
Average Case	O(log log n)	Data is uniformly distributed
Worst Case	O(n)	Data is not uniformly distributed (e.g., exponential)

Interpolation Search

Concept

Interpolation search is an improved variant of binary search designed for sorted, uniformly distributed arrays. It estimates where the target likely resides, rather than always checking the middle.

Analogy	Like how you open a dictionary: looking for 'Zoo', you open near the end; for 'Ant', near the front.

Probing Formula

```
pos = low + floor( (key − A[low]) × (high − low) / (A[high] − A[low]) )
```

A[low..high] = sorted array, key = target value, pos = estimated probe index.

Algorithm Pseudocode

```
INTERPOLATION-SEARCH(A, n, key)
  low ← 0,  high ← n − 1
  while low ≤ high and key ≥ A[low] and key ≤ A[high] do
    pos ← low + floor( (key−A[low])×(high−low) / (A[high]−A[low]) )
    if A[pos] == key then
      return pos   // found
    else if A[pos] < key then
      low ← pos + 1
    else
      high ← pos − 1
  return -1  // not found
```

Complexity

Case	Complexity	Condition
Best Case	O(1)	Key is at first probed position
Average Case	O(log log n)	Uniformly distributed data
Worst Case	O(n)	Exponentially distributed or skewed data

vs Binary Search: Binary = O(log n) always. Interpolation = O(log log n) on uniform data but degrades to O(n) on skewed distributions. Use Interpolation Search only when data is known to be uniformly distributed.

Worked Trace: key = 18

Array A = {2, 5, 8, 12, 16, 18, 23, 29, 31, 35, 42} (indices 0–10), key = 18.

Iter	low	high	A[low]	A[high]	Probe formula	pos	A[pos]	Action
1	0	10	2	42	0+floor((18-2)×10/40)	4	16	16<18 → low=5
2	5	10	18	42	5+floor((18-18)×5/24)	5	18	Match! Return 5

Result: key 18 found at index 5 in 2 probes. Binary search would need 3 probes.

3.2 Pattern Search — Overview

In pattern matching, we have a TEXT T of length n and a PATTERN P of length m. We want to find all positions where P appears in T.

Figure 3.2: The Pattern Matching Problem

```
TEXT    T : a b c a b c a b c d
```

```
PATTERN P :     a b c d

We slide P over T, one character at a time, and check for match.

Position 0:  a b c a  —  mismatch at T[3]='a' vs P[3]='d'
Position 1:  b c a b  —  mismatch at T[1]='b' vs P[0]='a'
Position 2:  c a b c  —  mismatch at T[2]='c' vs P[0]='a'
Position 3:  a b c d  —  MATCH ✓ at position 6
```

3.3 Naive String Matching Algorithm

The simplest approach: slide the pattern over the text one position at a time and check for a match at every position.

Algorithm

```
Algorithm NaiveStringMatch(T, P)
  n ← length(T)
  m ← length(P)
  for s ← 0 to n - m do
      if P[1..m] = T[s+1..s+m] then
          print 'Pattern found at shift', s
```

Dry Run Example

Figure 3.3: Naive Algorithm Dry Run

```
T = "AABCAABXAAB"   P = "AAB"   n=11, m=3
──────────────────────────────────────────────────────
s=0 : T[0..2]='AAB'  vs P='AAB'  → MATCH ✓ at position 0
s=1 : T[1..3]='ABC'  vs P='AAB'  → Mismatch at T[2]='B'
s=2 : T[2..4]='BCA'  vs P='AAB'  → Mismatch at T[2]='B'
s=3 : T[3..5]='CAA'  vs P='AAB'  → Mismatch at T[3]='C'
s=4 : T[4..6]='AAB'  vs P='AAB'  → MATCH ✓ at position 4
s=5 : T[5..7]='ABX'  vs P='AAB'  → Mismatch
s=6 : T[6..8]='BXA'  vs P='AAB'  → Mismatch
s=7 : T[7..9]='XAA'  vs P='AAB'  → Mismatch
s=8 : T[8..10]='AAB' vs P='AAB'  → MATCH ✓ at position 8
──────────────────────────────────────────────────────
Matches found at positions: 0, 4, 8
```

Analysis	Complexity	Explanation
Best Case	O(n)	Mismatch at first character every time
Worst Case	O((n-m+1)·m)	All characters match except last (e.g., T='AAA...A', P='AA...AB')

Limitation: Very slow for worst case. We need smarter algorithms that avoid redundant comparisons.

Traced Example: P = "AABA" in T = "AABAACAADAABAABA"

n=16, m=4. Algorithm slides P one position at a time:

Shift s	T window	Comparisons	Result

0	AABA	4	Match at s=0
1	ABAA	2	Mismatch at pos 2
2	BAAC	1	Mismatch at pos 1
3	AACA	3	Mismatch at pos 3
4	ACAA	2	Mismatch at pos 2
5	CAAD	1	Mismatch at pos 1
6	AADA	3	Mismatch at pos 3
7	ADAA	2	Mismatch at pos 2
8	DAAB	1	Mismatch at pos 1
9	AABA	4	Match at s=9
10	ABAA	2	Mismatch
11	BAAB	1	Mismatch
12	AABA	4	Match at s=12

SIRAGU

3.4 Rabin-Karp Algorithm

Rabin-Karp uses HASHING to speed up comparisons. Instead of comparing characters one by one, it computes a hash value for the pattern and each window of the text, comparing hashes first.

Core Idea

- Compute hash(P) once.
- Slide a window of size m over T, computing hash of each window.
- If hash(window) = hash(P): do a full character comparison (to avoid spurious hits).
- Use rolling hash to update the window hash in O(1) time.

Rolling Hash Formula

Hash Formula: h(s) = (d·h(s-1) - d^m · T[s-1] + T[s+m-1]) mod q

where d = number of characters in alphabet (e.g., 26 or 256), q = a large prime.

Algorithm

```
Algorithm RabinKarp(T, P, d, q)
  n ← length(T);  m ← length(P)
  h ← d^(m-1) mod q
  Compute p_hash = hash(P[0..m-1])
  Compute t_hash = hash(T[0..m-1])
  for s ← 0 to n-m do
      if p_hash = t_hash then
          if P[0..m-1] = T[s..s+m-1] then
              print 'Match at', s
      if s < n-m then
          t_hash = (d*(t_hash - T[s]*h) + T[s+m]) mod q
```

Worked Example

Figure 3.4: Rabin-Karp Rolling Hash Example

```
T = "2359023141526"   P = "31415"   d=10, q=13

p_hash = (3·10⁴ + 1·10³ + 4·10² + 1·10 + 5) mod 13
       = 31415 mod 13 = 7

Window T[0..4] = '23590'  → hash = 23590 mod 13 = 8  ≠ 7
Window T[1..5] = '35902'  → rolling hash update → ≠ 7
...
Window T[6..10]= '31415'  → hash = 7 = 7  → Full check → MATCH ✓

Rolling hash avoids recomputing from scratch each time → O(1) update
```

Case	Time Complexity	Reason
Best / Average	O(n + m)	Hash comparisons are O(1); no many spurious hits
Worst Case	O(nm)	Hash collisions cause full verification every step

Advantage: Rabin-Karp extends easily to 2D pattern matching and multi-pattern search.

Worked Example 1: P = "31" in T = "3141592653589793" (q=11)

n=16, m=2. d=10. h = 10^1 mod 11 = 10. Pattern hash: (3×10+1) mod 11 = 31 mod 11 = 9.

Shift s	Window	t_hash	= 9?	Actual match?
0	31	31 mod 11 = 9	Yes	3=3, 1=1 → MATCH ✓
1	14	7	No	—
2	41	4	No	—
3	15	6	No	—
4	59	3	No	—
5	92	8	No	—
6	26	7	No	—
7	65	3	No	—
8	53	9	Yes	5≠3 → SPURIOUS HIT
9	35	2	No	—
10	58	3	No	—
11	89	1	No	—
12	97	9	Yes	9≠3 → SPURIOUS HIT
13	79	2	No	—
14	93	5	No	—

Result: Match at s=0. Spurious hits at s=8 and s=12. One true match found.

3.5 Knuth-Morris-Pratt (KMP) Algorithm

KMP is the smartest of the three. It pre-processes the PATTERN to build a Failure Function (also called the partial match table or lps array). This tells us, after a mismatch, how far we can jump without re-examining already matched characters.

Key Idea

When a mismatch occurs at position j in the pattern after matching i characters, we do NOT restart from the beginning. Instead, we use the lps[] (Longest Proper Prefix which is also Suffix) array to skip ahead.

Step 1: Build the LPS (Failure Function) Array

For pattern P, lps[i] = length of the longest proper prefix of P[0..i] that is also a suffix.

Figure 3.5: Building the LPS Array

```
Pattern P = A B C D A B C
Index       0 1 2 3 4 5 6

lps[0] = 0  (single char, no proper prefix)
lps[1] = 0  'AB' → no prefix = suffix
lps[2] = 0  'ABC' → no prefix = suffix
lps[3] = 0  'ABCD' → no prefix = suffix
lps[4] = 1  'ABCDA' → 'A' is prefix AND suffix → length 1
lps[5] = 2  'ABCDAB' → 'AB' is prefix AND suffix → length 2
lps[6] = 3  'ABCDABC' → 'ABC' is prefix AND suffix → length 3

LPS = [0, 0, 0, 0, 1, 2, 3]
```

Step 2: KMP Search Algorithm

```
Algorithm KMPSearch(T, P)
  n ← length(T);  m ← length(P)
  lps ← ComputeLPS(P)
  i ← 0  (index for T)
  j ← 0  (index for P)
  while i < n do
      if T[i] = P[j] then
          i++; j++
      if j = m then
          print 'Match at', i - m
          j ← lps[j-1]
      else if i < n and T[i] ≠ P[j] then
          if j ≠ 0 then j ← lps[j-1]
          else i++
```

KMP Dry Run — Complete Trace

Figure 3.6: KMP Complete Dry Run

```
T = "AABCAABXAABC"   P = "AABC"   lps = [0,1,0,0]
──────────────────────────────────────────────────

i=0,j=0: T[0]='A'=P[0]='A'  → i=1, j=1
i=1,j=1: T[1]='A'=P[1]='A'  → i=2, j=2
```

```
i=2,j=2: T[2]='B'=P[2]='B'  → i=3, j=3
i=3,j=3: T[3]='C'=P[3]='C'  → i=4, j=4  j=m → MATCH at 0 ✓
         j ← lps[3] = 0
i=4,j=0: T[4]='A'=P[0]='A'  → i=5, j=1
i=5,j=1: T[5]='A'=P[1]='A'  → i=6, j=2
i=6,j=2: T[6]='B'=P[2]='B'  → i=7, j=3
i=7,j=3: T[7]='X'≠P[3]='C'  → j ← lps[2]=0
i=7,j=0: T[7]='X'≠P[0]='A'  → i=8
i=8,j=0: T[8]='A'=P[0]='A'  → i=9, j=1
i=9,j=1: T[9]='A'=P[1]='A'  → i=10,j=2
i=10,j=2:T[10]='B'=P[2]='B' → i=11,j=3
i=11,j=3:T[11]='C'=P[3]='C' → i=12,j=4 j=m → MATCH at 8 ✓
──────────────────────────────────────────────────────────
Matches at positions: 0, 8
```

How KMP Skips Redundant Comparisons

Figure 3.7: Naive vs KMP — Skip Comparison

```
Naive approach after mismatch:          KMP after mismatch:
──────────────────────────────          ──────────────────────
T: A B C D A B C E                      T: A B C D A B C E
   A B C D A B C                           A B C D A B C
Mismatch at E vs nothing                KMP uses lps to shift
Restart from beginning of P             Only shift to safe position
Wasted: 6 comparisons                   No wasted comparisons!
```

Phase	Time Complexity	Explanation
LPS Computation	O(m)	Scans pattern once
Search Phase	O(n)	Never backtracks in text T
Overall KMP	O(n + m)	Optimal for single-pattern search

8.2 Failure Function Algorithm

```
COMPUTE-FAILURE-FUNCTION(P)
  m ← |P|
  π[0] ← 0   // always 0 by definition
  k ← 0
  for i ← 1 to m−1 do
    while k > 0 and P[k] ≠ P[i] do
      k ← π[k−1]   // fall back
    if P[k] == P[i] then
      k ← k + 1
    π[i] ← k
  return π
```

Failure Functions for All Exam Patterns

P = "ABABCABAB"

Index i	0	1	2	3	4	5	6	7	8
P[i]	A	B	A	B	C	A	B	A	B
π[i]	0	0	1	2	0	1	2	3	4

- i=2: "ABA" → prefix "A" = suffix "A" → π=1
- i=3: "ABAB" → prefix "AB" = suffix "AB" → π=2
- i=4: "ABABC" → no match → π=0
- i=7: "ABABCABA" → "ABA" matches → π=3
- i=8: "ABABCABAB" → "ABAB" matches → π=4

P = "AABAAB"

Index i	0	1	2	3	4	5
P[i]	A	A	B	A	A	B
π[i]	0	1	0	1	2	3

P = "AABAABAAA"

Index i	0	1	2	3	4	5	6	7	8
P[i]	A	A	B	A	A	B	A	A	A
π[i]	0	1	0	1	2	3	4	5	2

P = "ABCDABD"

Index i	0	1	2	3	4	5	6
P[i]	A	B	C	D	A	B	D
π[i]	0	0	0	0	1	2	0

P = "ALGORITHM" (Plagiarism Detection)

Index i	0	1	2	3	4	5	6	7	8
P[i]	A	L	G	O	R	I	T	H	M
π[i]	0	0	0	0	0	0	0	0	0

All zeros — no prefix of ALGORITHM is also a suffix. KMP behaves like Naive here but never backtracks in T.

P = "1011101" (Network Intrusion Detection)

i	0	1	2	3	4	5	6
P[i]	1	0	1	1	1	0	1
π[i]	0	0	1	1	1	2	3

```
Build step-by-step:
i=0: π[0]=0 (by rule)
i=1: P[0]=1 ≠ P[1]=0, k stays 0 → π[1]=0
i=2: P[0]=1 = P[2]=1, k→1 → π[2]=1
i=3: P[1]=0 ≠ P[3]=1 → k=π[0]=0; P[0]=1=P[3]=1 → k→1 → π[3]=1
i=4: P[1]=0 ≠ P[4]=1 → k=0; P[0]=1=P[4]=1 → k→1 → π[4]=1
i=5: P[1]=0=P[5]=0 → k→2 → π[5]=2
i=6: P[2]=1=P[6]=1 → k→3 → π[6]=3
```

Verification for π[6]=3: longest proper prefix of '1011101' that equals a suffix is '101' (appears at both start and end) → length 3 ✓

P = "SEARCH" (Autocomplete)

Index i	0	1	2	3	4	5
P[i]	S	E	A	R	C	H
π[i]	0	0	0	0	0	0

All zeros — all characters distinct. No prefix equals any suffix.

SIRAGU

SECTION 4: COMPARISON OF STRING MATCHING ALGORITHMS

Feature	Naive	Rabin-Karp	KMP
Preprocessing	None	O(m) — compute hash	O(m) — build lps[]
Matching Time	O((n-m+1)·m)	O(n+m) avg, O(nm) worst	O(n)
Space	O(1)	O(1)	O(m)
Backtracking in T	Yes	No	No
Multiple Patterns	Separate runs needed	Efficient extension	Not direct
Best Used When	Small n, simple needs	Multiple patterns, 2D	Single pattern, large text

Figure 4.1: Algorithm Selection Guide

```
ALGORITHM SELECTION GUIDE
─────────────────────────────────────────────────────

n is small (< 1000)?
    └─ Use NAIVE — simple to implement

Searching for multiple patterns simultaneously?
    └─ Use RABIN-KARP — easily extended

Large text, single pattern, need guaranteed O(n)?
    └─ Use KMP — optimal linear time guarantee
─────────────────────────────────────────────────────
```

SIRAGU

SECTION 5: QUICK REVISION — KEY FORMULAS

Concept	Formula / Rule
Big-O	$T(n) = O(f(n))$ iff $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$
Big-Omega	$T(n) = \Omega(f(n))$ iff $T(n) \geq c \cdot f(n)$ for all $n \geq n_0$
Theta	$T(n) = \Theta(f(n))$ iff $c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$
Binary Search Recurrence	$T(n) = T(n/2) + 1 \rightarrow O(\log n)$
Merge Sort Recurrence	$T(n) = 2T(n/2) + n \rightarrow O(n \log n)$
Interpolation Search Position	pos = low + [(key-A[low])/(A[high]-A[low])] × (high-low)
Rabin-Karp Rolling Hash	h(s) = (d·h(s-1) - d^m · T[s-1] + T[s+m]) mod q
KMP LPS	lps[i] = length of longest proper prefix of P[0..i] = suffix

SECTION 6: IMPORTANT QUESTIONS

Short Answer (2 marks)

1. Define algorithm. List its properties.
2. What is the difference between time and space complexity?
3. Define Big-O, Omega, and Theta notations.
4. State the reflexivity and transitivity properties of asymptotic notation.
5. What is a recurrence relation? Give an example.
6. What is interpolation search? When does it outperform binary search?
7. What is a spurious hit in the Rabin-Karp algorithm?
8. Define the failure function (lps) in KMP.

Long Answer (13 marks)

1. Explain the naive string-matching algorithm with example. Analyse its time complexity.
2. Describe the Rabin-Karp algorithm with a complete worked example. Compare it with the naive method.
3. Explain the KMP algorithm. Show how the lps array is built and how the search works with a trace for T='ABABDABACDABABCABAB' and P='ABABCABAB'.
4. Explain all three asymptotic notations with diagrams and examples. State their properties.
5. Solve T(n) = 2T(n/2) + n using the substitution method and show the recursion tree.

SIRAGU

— End of Unit I Lecture Notes —